

Go Back

Hardware

Nano 33 BLE

Tutorials

Controlling RGB LED
Through Bluetooth®

Getting Started with
OpenMV

Connecting Two Nano 33
BLE Boards Through I2C

Accessing Accelerometer
Data on Nano 33 BLE

Accessing Gyroscope Data
on Nano 33 BLE

Accessing Magnetometer
Data on Nano 33 BLE

Debugging with Lauterbach
TRACE32 GDB Front-End
Debugger

Connecting Two Nano 33
BLE Boards Through UART

Home / Hardware / Nano 33 BLE /
Connecting Two Nano 33 BLE Boards
Through I2C



This tutorial refers to a
product that has reached its
end-of-life status.

Connecting Two Nano 33 BLE Boards Through I2C

Learn how to send data from the Nano 33
BLE board to another board via I2C.

Author · Benjamin · Last
Dannegård · revision · 07/17/2024

ON THIS PAGE

Goals

- Hardware & Software Needed
- I2C Protocol
- The Wire Library +
- Creating the Program +
- Testing It Out +
- Conclusion

In this tutorial we will control the built-in LED of an Arduino Nano 33 BLE from another Arduino Nano 33 BLE. To do so, we will connect both boards using a wired communication protocol called I2C.

Note: This example would work connecting an Arduino Bluetooth® Low Energy board with any other Arduino board, but be mindful that by using any other board the connections might differ. If the connections vary, it may happen that the code might also need changes to match with the I2C pins of the different boards.

In this example, we will power both the Arduino boards through a computer, then through the Serial Monitor, we will introduce some commands to turn **ON** or **OFF** the LED of the 33 BLE board.

Goals

The goals of this project are:

- Understand what I2C protocol is.

- Use the `Wire.h` library.

- Use I2C communication between two Arduino boards.

Hardware & Software Needed

For this project we will need:

- Arduino Nano 33 BLE

- Arduino Nano 33 BLE (or any other Arduino board that

10 x [jumper wires](#)

2 x 4.7k Ohm resistor

I2C Protocol

I2C (Inter Integrated Circuits) is a well known and widely used protocol. It allows us to connect multiple readers to a single writer, meaning that, you could have more than one sensor connected to the same pins of your Arduino board through I2C. The only requisite is that each of those sensors have a unique address.

The I2C protocol involves using two lines to send and receive data: a serial clock pin (SCL) where the Arduino writer board pulses at a regular interval, and a serial data pin (SDA) over which data is sent between the two devices.

As the clock line changes from **low** to **high**, a single bit of information that will form the address of a specific device, and a command or data is transferred from the board to the I2C device over the SDA line. When this information is sent, the called upon device executes the request and transmits it's data back, if required, to the board over the same line using the clock signal still generated by the writer on SCL.

If you want to learn more about the I2C protocol and how it works, you can check the [I2C standards](#).

The Wire Library

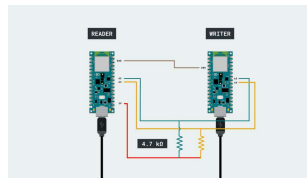
To simplify the use of the I2C

implement and use this protocol throughout different Arduino boards. This library can only be used on some specific pins of the board, those pins are called SCL and SDA, on the Arduino Nano family boards, the SDA (data line) and SCL (clock line) are on the A4 and A5 pin headers respectively.

If you want to read more about the [Wire.h](#) library see [here](#).

Circuit

In order to communicate both Arduino boards, we will need to connect them as shown in the image below.



Reader-Writer circuit,
connected with wires and
resistors.

At the moment of making the connections, we need to remember that this protocol has two dedicated lines SCL (clock) and SDA (data) so we need to make sure to connect the SCL pin, of one of the boards, with the SCL of the other one. The same goes for the SDA as it is shown in the image.

To finish, we need to connect the GND pins of both boards to each other.

Note: In order to enable serial communication, both Arduino boards must be connected to your

Creating the Program

1. Setting up

First start by connecting your Arduino Nano 33 BLE board to the computer and opening the Arduino Create Cloud Editor. This board will act as the reader. Start a new sketch and name it **Nano_I2C_Reader**.

2. Reader code explanation

Let's start by adding the `Wire.h` library by adding the `#include <Wire.h>` statement. Then, in the `setup()` we need to add `Wire.begin(8)` to initiate the library and give the reader board the address of 8, so the writer can identify who to send or receive data from. We will also use `Wire.onReceive()` which gets called every time data is received from the writer.

```
1  #include <Wire.h>
2
3  void setup() {
4      Wire.begin(8);
5      Wire.onReceive(receiveE
6  }
```

The `loop()` of this code is not as important, since nothing is required to run until we receive something through I2C. Due to this, we will leave the `loop()` function empty.

Instead the focus is on the `loop()` where we will create a

received by the board. We can store the data we will receive in the `c` variable, for ease of use. The data we will receive will be a character, so we need to compare it as such. The `if` statements will check if the character that was sent is a 1 or a 0. It will then turn **on** or **off** the built-in LED depending on the received character.

```
1 void receiveEvent(int how
2   char c = Wire.read();
3   if(c == '0'){
4     digitalWrite(LED_BUI
5   }
6   if(c == '1'){
7     digitalWrite(LED_BUI
8   }
9 }
```

3. Uploading the code to the reader

Once we have finished the code, let's upload it to our Arduino Nano 33 BLE board. This board is now programmed to act as the reader in this scenario. Once the code is uploaded, let's connect the other board to the computer.

Note: After uploading the code to the reader board, it is not necessary for it to stay connected to the computer in order to function. However, it needs to be powered in order to work.

4. Writer code

It is time to open a new sketch and name it as **Nano_I2C_Writer**, then let's add the Wire library as we did in

In the `setup()` we need to add the `Serial.begin(9600)` which opens the serial port and sets the data rate to 9600 bits per second (baud). Next we will add the `while (!Serial);` which ensures that there is something at the other end of the USB connection, for the Arduino to talk to before it starts sending messages. Otherwise, the message might be sent, but not displayed. We also add `Wire.begin()` to initiate the Wire library and join the I2C bus as a writer or reader.

```
1  #include <Wire.h>
2
3  void setup() {
4      Serial.begin(9600);
5      while (!Serial);
6      Wire.begin(); // join i
7  }
```

Next, in the `loop()` we first print the message that lets us know what values can be entered. We then define a variable, `ledVal`, that will hold the value we entered into the Serial Monitor. With the `readSerial()` we will take the value entered into the Serial Monitor and store it in the `ledVal` variable. We will then begin the I2C communication.

With `Wire.beginTransmission(8)` we designate which reader we will send data to. If you remember, in the reader's code we gave the board the address 8. After communication has been opened we use `Wire.write()` to chose what we send to the reader, in this case we send the variable `ledVal`. After

that we simply end the transmission with `Wire.endTransmission()`.

```
1 void loop() {
2   Serial.print("Enter 0
3   char ledVal[0];
4   readSerial(ledVal);
5   Serial.println(ledVal)
6   Wire.beginTransaction
7   Wire.write(ledVal);
8   Wire.endTransmission()
9   delay(500);
10 }
```

The `readSerial()` function allows us to take a value that is entered into the Serial Monitor and store it into a variable, enabling us to send it via I2C. The function detects when an entry is sent and takes whatever is typed before the entry and puts it into a variable. It can hold longer strings of characters if needed.

```
1 int readSerial(char resu
2   int i = 0;
3   while (1) {
4     while (Serial.availa
5       char inChar = Seri
6       if (inChar == '\n'
7         result[i] = '\0'
8         Serial.flush();
9         return 0;
10      }
11      if (inChar != '\r'
12        result[i] = inCh
13        i++;
14      }
15    }
16  }
17 }
```

5. Uploading the code to the writer

Once we have finished the code, let's upload it to our Arduino Nano 33 BLE board. This board is now programmed to act as the writer.

6. Complete code

If you choose to skip the code building section, the complete code for both the reader and the writer can be found below:

Reader:

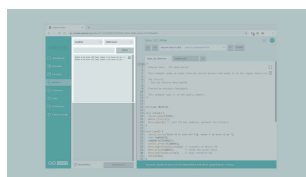
```
1  #include <Wire.h>
2
3  void setup() {
4      Wire.begin(8);
5      Wire.onReceive(receive
6  }
7
8  void loop() {
9      delay(100);
10 }
11
12 void receiveEvent(int ho
13     char c = Wire.read();
14     if(c == '0'){
15         digitalWrite(LED_BU
16     }
17     if(c == '1'){
18         digitalWrite(LED_BU
19     }
20 }
```

Writer:

```
1  #include <Wire.h>
2
3  void setup() {
4      Serial.begin(9600);
5      while (!Serial);
6      Wire.begin(); // join
7  }
8
9  void loop() {
10     Serial.print("Enter 0
11     char ledVal[0];
12     readSerial(ledVal);
13     Serial.println(ledVal
14     Wire.beginTransmissio
15     Wire.write(ledVal);
16     Wire.endTransmission(
17     delay(500);
18 }
19
20 /* Read input serial */
21 int readSerial(char res
22     int i = 0;
23     while (1) {
24         while (Serial.avail
25             char inChar = Ser
26             if (inChar == '\n
27                 result[i] = '\0
28             Serial.flush();
```

Testing It Out

After you have successfully verified and uploaded the sketch to the two boards, make sure the writer board is connected, open the Serial Monitor. You will see a text asking for a value of 1 or 0 to be entered. When a value is entered it will be sent to the reader board.



Output from Serial
Monitor.

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

Missing a bracket or a semicolon.

Arduino board connected to the wrong port.

Connection between the Arduino boards are not correct.

Accidental interruption of cable connection.

Conclusion

In this simple tutorial we learned how to connect two boards so that they can communicate using I2C, with the help of the **Wire.h** library.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

